

✓ Cross-validation

Splitting

? Question

Learn more in [Cross-validation tutorial](#).

To pick a fixed number of windows and optimize the window length such that they collectively cover the maximum amount of the index while keeping the train or test set non-overlapping, use `Splitter.from_n_rolling` with `length="optimize"`. Under the hood, it minimizes any empty space using SciPy.

Pick longest 20 windows for WFA such that test ranges don't overlap

```
splitter = vbt.Splitter.from_n_rolling(
    data.index,
    n=20,
    length="optimize",
    split=0.7, ❶
    optimize_anchor_set=1, ❷
    set_labels=["train", "test"]
)
```

- ❶ 70% for training, 30% for testing
- ❷ Make the test set non-overlapping. Change to 0 for the train set.

———— + ————

When using `Splitter.from_rolling` and the last window doesn't fit, it will be removed, leaving a gap on the right-hand side. To remove the oldest window instead, use `backwards="sorted"`.

Roll a window that fills more recent data and with no gaps between test sets

```
length = 1000
ratio = 0.95
train_length = round(length * ratio)
test_length = length - train_length

splitter = vbt.Splitter.from_rolling(
    data.index,
    length=length,
    split=train_length,
    offset_anchor_set=None,
    offset=-test_length,
    backwards="sorted"
)
```

———— + ————

To create a gap between the train set and the test set, use `RelRange` with `is_gap=True`.

Roll an expanding window with a variable train set, a gap of 10 rows, and a test set of 20 rows

```
splitter = vbt.Splitter.from_expanding(
    data.index,
    min_length=130,
    offset=10, ❶
    split=(1.0, vbt.RelRange(length=10, is_gap=True), 20),
    split_range_kwargs=dict(backwards=True) ❷
)
```

- ❶ Shift each split by the same number of rows as in the gap
- ❷ Split each range by first calculating the test set and only then the train set, otherwise `1.0` (100%) will be calculated first and will take the entire split

———— + ————

To roll a time-periodic window, use `Splitter.from_ranges` with `every` and `lookback_period` arguments as date offsets.

Reserve 3 years for training and 1 year for testing

```
splitter = vbt.Splitter.from_ranges(
    data.index,
    every="Y",
    lookback_period="4Y",
    split=(
        vbt.RepEval("index.year != index.year[-1]"), ❶
        vbt.RepEval("index.year == index.year[-1]") ❷
    )
)
```

- ❶ Training set should include all years in the split apart from the last one

- ② Test set should include the last year in the split only

Taking

To split an object along the index (time) axis, we need to create a **Splitter** instance and then "take" chunks from that object.

How to split an object in two lines

```
splitter = vbt.Splitter.from_n_rolling(data.index, n=10)
data_chunks = splitter.take(data) ①

# -----

splitter = vbt.Splitter.from_ranges(df.index, every="W")
new_df = splitter.take(df, into="reset_stacked") ②
```

- ① VBT object
- ② Regular array

———— + ————

Also, most VBT objects have a `split` method that can combine these both operations into one. The method will determine the correct splitting operation automatically based on the supplied arguments.

How to split an object in one line

```
data_chunks = data.split(n=10) ①

# -----

new_df = df.vbt.split(every="W") ②
```

- ① VBT object. Method `from_n_rolling` is guessed based on `n=10`.
- ② Regular array. Method `from_ranges` is guessed based on `every="W"`. Option `into="reset_stacked"` is enabled automatically.

Testing

To cross-validate a function that takes only one parameter combination at a time on a grid of parameter combinations, use `@vbt.cv_split`. It's a combination of `@vbt.parameterized` (which takes a grid of parameter combinations and runs a function on each combination while merging the results) and `@vbt.split` (which runs a function on each split and set combination).

Cross-validate a function to maximize the Sharpe ratio

```
def selection(grid_results): ①
    return vbt.LabelSel([grid_results.idxmax()]) ②

@vbt.cv_split(
    splitter="from_n_rolling", ③
    splitter_kwargs=dict(n=10, split=0.5, set_labels=["train", "test"]), ④
    takeable_args=["data"], ⑤
    execute_kwargs=dict(), ⑥
    parameterized_kwargs=dict(merge_func="concat"), ⑦
    merge_func="concat", ⑧
    selection=vbt.RepFunc(selection), ⑨
    return_grid=False ⑩
)
def my_pipeline(data, param1_value, param2_value): ⑪
    ...
    return pf.sharpe_ratio

cv_sharpe_ratios = my_pipeline( ⑫
    data,
    vbt.Param(param1_values),
    vbt.Param(param2_values)
)

# -----

@vbt.cv_split(..., takeable_args=None) ⑬
def my_pipeline(range_, data, param1_value, param2_value):
    data_range = data.iloc[range_]
    ...
    return pf.sharpe_ratio

cv_sharpe_ratios = my_pipeline(
    vbt.Rep("range_"),
    data,
    vbt.Param([1, 2, 3]),
    vbt.Param([1, 2, 3]),
    _index=data.index ⑭
)
```

- ① Function that returns the index of the best parameter combination
- ② Find the parameter combination of the highest Sharpe ratio. Wrap with `LabelSel` to tell vectorbtpro that the returned value is a label and not a position in case it's an integer. Also, wrap the value with a list to show the parameter combination in the final index.

- ③ Name of the splitting method, such as `Splitter.from_n_rolling`
- ④ Keyword arguments passed to the splitting method
- ⑤ Name of the arrays that should be split. You can also pass `vbt.Takeable(data)` directly to the function instead.
- ⑥ Keyword arguments passed to `execute` to control the execution of split and set combinations
- ⑦ Keyword arguments passed to the `@vbt.parameterized`. Here we want to concatenate the results of all the parameter combinations into a single Pandas Series.
- ⑧ Function (name) to merge all the split and set combinations. Here we want to concatenate all the Pandas Series into a single Pandas Series.
- ⑨ Either a specific index or a template to pick the best parameter combination
- ⑩ Whether to return the entire grid of parameter combinations and not only the best ones
- ⑪ Similarly to `@vbt.parameterized`, VBT will pass only one parameter combination at a time as single values. Any takeable argument (here `data`) will contain only values that correspond to the current split and set combination.
- ⑫ The returned Pandas Series will contain the cross-validation results by split and set combination
- ⑬ Same as above but select the date range manually in the function
- ⑭ Any argument that is meant to be passed to the decorator can be also passed directly to the function by prepending the underscore

———— + ————

To skip a parameter combination, return `NoResult`. This may be helpful to exclude a parameter combination that raises an error. `NoResult` can be also returned by the selection function to skip the entire split and set combination. Once excluded, the combination won't be visible in the final index.

Skip split and set combinations where there are no satisfactory parameters

```

1
def selection(grid_results):
    sharpe_ratio = grid_results.xs("Sharpe Ratio", level=-1).astype(float)
    return vbt.LabelSel([sharpe_ratio.idxmax()])

@vbt.cv_split(...)
def my_pipeline(...):
    ...
    stats_sr = pf.stats(agg_func=None)
    if stats_sr["Min Value"] > 0 and stats_sr["Total Trades"] >= 20: ②
        return stats_sr
    return vbt.NoResult

# -----
3
def selection(grid_results):
    sharpe_ratio = grid_results.xs("Sharpe Ratio", level=-1).astype(float)
    min_value = grid_results.xs("Min Value", level=-1).astype(float)
    total_trades = grid_results.xs("Total Trades", level=-1).astype(int)
    sharpe_ratio = sharpe_ratio[(min_value > 0) & (total_trades >= 20)]
    if len(sharpe_ratio) == 0:
        return vbt.NoResult
    return vbt.LabelSel([sharpe_ratio.idxmax()])

@vbt.cv_split(...)
def my_pipeline(...):
    ...
    return pf.stats(agg_func=None)

```

- ① Filter parameter combinations on the fly and then select the best ones from those that left (if any)
- ② Keep the parameter combination only if the minimum portfolio value is greater than 0 (i.e., the position hasn't been liquidated) and the number of trades is 20 or greater
- ③ Same as above but return all parameter combinations and then filter them in the selection function

———— + ————

To warm up one or more indicators, instruct VBT to pass a date range instead of selecting it from data, and prepend a buffer to this date range. Then, manually select this extended date range from the data and run your indicators on the selected date range. Finally, remove the buffer from the indicator(s).

Warm up a SMA crossover

```

@vbt.cv_split(..., index_from="data")
def buffered_sma_pipeline(data, range_, fast_period, slow_period, ...):
    buffer_len = max(fast_period, slow_period) ①
    buffered_range = slice(range_.start - buffer_len, range_.stop) ②
    data_buffered = data.iloc[buffered_range] ③

    fast_sma_buffered = data_buffered.run("sma", fast_period, hide_params=True)
    slow_sma_buffered = data_buffered.run("sma", slow_period, hide_params=True)
    entries_buffered = fast_sma_buffered.real_crossed_above(slow_sma_buffered)
    exits_buffered = fast_sma_buffered.real_crossed_below(slow_sma_buffered)

    data = data_buffered.iloc[buffer_len:] ④
    entries = entries_buffered.iloc[buffer_len:]
    exits = exits_buffered.iloc[buffer_len:]
    ...

```

```
buffered_sma_pipeline(  
    data, ⑤  
    vbt.Rep("range_"), ⑥  
    vbt.Param(fast_periods, condition="x < slow_period"),  
    vbt.Param(slow_periods),  
    ...  
)
```

- ① Determine the length of the buffer
- ② Extend the date range with the buffer
- ③ Select the data that corresponds to the extended date range
- ④ After running all the indicators, remove the buffer
- ⑤ Pass data as it is (without selection)
- ⑥ Instruct VBT to pass the date range as a slice